Scrum Poker – Leitfaden fürs Fachgespräch

Im Fachgespräch wird **jedes Gruppenmitglied einzeln** zu beliebigen Stellen im Code befragt. Wer nichts erklären kann, bekommt die Note 1.0 – unabhängig davon, wie gut die App ist. Weil die App grösstenteils mit KI erstellt wurde (siehe Dokumentation, Kapitel 15 «KI-Einsatz»), müsst ihr den Code jetzt so gut kennen, als hättet ihr ihn selbst geschrieben.

So arbeitet ihr mit diesem Leitfaden

1. Teil A (Rundgang) mit dem Code daneben lesen. Jede Datei öffnen und die genannten Zeilen ansehen.
2. Teil B (Musterantworten) laut üben, **in eigenen Worten**, ohne abzulesen. Eine Person fragt, die andere zeigt die Stelle im Code und erklärt.
3. Teil C (weitere Fragen) als Quiz verwenden.
4. Teil E (Übungsaufgaben) wirklich durchführen: Code ändern, beobachten, mit `git checkout .` zurücksetzen. Wer etwas selbst kaputt gemacht und repariert hat, kann es erklären.
5. Teil D (Demo und Checkliste) am Vortag einmal komplett durchspielen.

Verweise wie `PokerRoom.tsx:82` bedeuten Datei und Zeilennummer. Pfade sind relativ zum Projektordner (im Code-ZIP der Ordner `scrum-poker/`).

Kursthemen, auf die die Spalte «Kursthema» verweist:

Kürzel	Thema
HTML/CSS	semantisches HTML, Box-Modell, Flexbox, Grid, Responsive Design, Barrierefreiheit
TS	JavaScript/TypeScript: Types, Interfaces, Union Types, Generics, `unknown`, Type Guards, `async` / `await`, Promises
React	Komponenten, JSX, Props, Listen mit `key`, Events, bedingtes Rendern, Komposition (`children`)
State	`useState`, Lifting State Up, Single Source of Truth, abgeleitete Werte, kontrollierte Formulare
Effects	`useEffect`, Cleanup, Dependencies, StrictMode, `useCallback`, `useRef`, Custom Hooks
Context	`createContext`, Provider, `useContext`, Context vs. Props
Routing	React Router: `Routes`, `Route`, `useParams`, `useNavigate`, `Link`, `Outlet`
HTTP	REST, HTTP-Methoden, Statuscodes, JSON, `fetch`, Fehlerbehandlung, Polling
Express	Node.js, Express, Router, Middleware, Error-Handler
Test	Vitest, React Testing Library, supertest, Mocks und Fake-Timer
Tooling	pnpm-Workspace, Vite, Proxy, ESLint, `tsconfig`

TODO Gruppe: Falls der Kurs nummerierte Blöcke hat (z. B. «Block 4: Hooks»), die Nummern hier in der Tabelle ergänzen.

Teil A – Rundgang durch alle Dateien

A.1 Projektwurzel

Datei	Zweck	Wichtig zu wissen	Kursthema
`package.json`	Skripte für alle Pakete	`dev` startet Server und Client mit `concurrently` gleichzeitig. `test` / `build` / `typecheck` laufen mit `pnpm --recursive` in jedem Paket. `engines.node >= 22.12`, `packageManager: pnpm@10.34.6` (Corepack nimmt genau diese Version). `test:e2e` startet Playwright.	Tooling
`pnpm-workspace.yaml`	macht `client`, `server`, `shared` zu einem Workspace	Darum kann der Client `@scrum-poker/shared` mit `"workspace:*"` einbinden, ohne es zu veröffentlichen.	Tooling
`pnpm-lock.yaml`	genaue Versionen aller Abhängigkeiten	Gleiche Installation auf jedem Rechner. Nie von Hand ändern.	Tooling
`eslint.config.js`	ESLint für alle Pakete	`no-explicit-any` ist ein Fehler. `argsIgnorePattern: '^_'` erlaubt unbenutzte Parameter mit `_` (der Express-Error-Handler braucht vier Parameter). Für `client/` zusätzlich die React-Hooks-Regeln (prüfen u. a. die Dependency-Arrays).	Tooling
`.gitignore`	was nicht ins Git gehört	`node_modules/`, `dist/`, `*.tsbuildinfo`, Playwright-Ergebnisse, das Abgabe-ZIP	Tooling
`playwright.config.ts`	Konfiguration der E2E-Tests	startet Server und Client selbst auf eigenen Ports (3101/5273) über `webServer`, Chromium, ein Worker (die Tests teilen sich einen Server). Bewusst der Dev-Server: Nur dort läuft StrictMode.	Test
`e2e/scrum-poker.spec.ts`	End-to-End-Tests	zwei Browser-Kontexte = zwei Personen. Prüft den ganzen Ablauf, Tab schliessen, Fehlerzustände, kein horizontales Scrollen und eine saubere Browser-Konsole.	Test
`e2e/tsconfig.json`	Typprüfung der E2E-Tests	wird von `pnpm typecheck` mitgeprüft (`tsc -p e2e`)	TS
`README.md`	Kurzanleitung		
`docs/FACHGESPRAECH.md` / `.pdf` / `.html`	dieser Leitfaden		
`DOKUMENTATION.md` / `.pdf` / `.html`	Projektdokumentation	wird neben dem Code-ZIP abgegeben, nicht darin	

Datei	Zweck	Wichtig zu wissen	Kursthema
`scripts/`, `deploy/`	`make-submission.sh` (Code-ZIP), `build-pdf.sh` + `pdf/` (PDF und HTML erzeugen), Betrieb auf dem Server	gehören nicht zur App und sind nicht im Code-ZIP	

A.2 `shared/` – der gemeinsame Vertrag

`shared/src/index.ts` – Types und Logik, die Client und Server beide importieren.

Zeilen	Was	Erklärung
8	`type DeckType = 'fibonacci' | 'tshirt'`	**Union aus String-Literalen.** Nur diese zwei Werte sind erlaubt, ein Tippfehler wie `'tshrit'` ist ein Compile-Fehler.
10	`type CardValue = string`	Kartenwerte sind Strings (`'5'`, `'XL'`, `'?'`). Gültig ist ein Wert nur, wenn er im aktiven Deck vorkommt. Das prüft der Server zur Laufzeit.
12-26	`Deck`, `DECKS`	`Record<DeckType, Deck>` erzwingt, dass es für **jeden** Deck-Typ einen Eintrag gibt. `readonly CardValue[]` verhindert, dass jemand die Karten verändert.
30-36	`TSHIRT_VALUES`	Abbildung XS=1 ... XL=8 für den Durchschnitt
38	`MAX_NAME_LENGTH = 30`	Eine Konstante für Client **und** Server. Ändert man sie, ändern sich beide Prüfungen.
44-51	`Participant`	Was der Client über eine Person sieht: `hasVoted` immer, `vote` nur nach dem Aufdecken (oder die eigene). `vote: CardValue | null` zwingt den Client, den Fall `null` zu behandeln.
53-58	`SessionState`	Antwort von `GET /api/sessions/:id` und allen Aktionen
60-65	`SessionSummary`	ein Eintrag der Liste. `createdAt` ist ein ISO-String, weil JSON keinen Datumstyp hat.
67-112	Request-/Response-Types	Pro Route ein Type, z. B. `CastVoteRequest { value: CardValue | null }`. Der Client baut damit den Body, der Server prüft ihn dagegen.
110-112	`ApiErrorBody { error: string }`	Form jeder Fehlerantwort
118-125	`RoundResult`	Ergebnis von `computeResult()`
127-134	`toNumber()`	T-Shirt → Tabelle, sonst `Number(card)`. `'?'` und `'☕'` ergeben `NaN` → `null`.
136-149	`closestTshirtSize()`	sucht die Grösse mit dem kleinsten Abstand. `<=` (Zeile 143) wählt bei Gleichstand die grössere.
151-168	`computeResult()`	**reine Funktion**: gleiche Eingabe → gleiche Ausgabe, keine Nebenwirkungen. Darum einfach testbar. `.filter((value) => value !== null)` – TypeScript erkennt seit Version 5.5, dass danach nur noch `number` übrig ist. Zeile 164 rundet auf eine Nachkommastelle.

`shared/src/computeResult.test.ts` – acht Unit-Tests für `computeResult()` (siehe DOKUMENTATION Kapitel 12). Kursthema: Test.

`shared/package.json`: `"exports": "./src/index.ts"` – das Paket zeigt direkt auf den TypeScript-Quelltext. Vite und `tsx` können TS direkt lesen, es braucht keinen Build-Schritt.
`shared/tsconfig.json`: `strict: true`, `noEmit` (nur Typprüfung).

A.3 `server/` – Express-API

`server/src/index.ts` (Kursthema: Express)

Zeile	Erklärung
5	`setInterval(() => removeInactiveParticipants(Date.now()), CLEANUP_INTERVAL_MS)` – alle 5 s Karteileichen entfernen. `Date.now()` wird übergeben, damit die Funktion in Tests mit einer festen Zeit aufgerufen werden kann.
7-9	`app.listen(PORT, …)` startet den HTTP-Server auf Port 3001.

`server/src/app.ts` – die Express-App **ohne** `listen()`. So können die Tests die App mit supertest direkt aufrufen, ohne einen Port zu belegen. (Kursthema: Express)

Zeile	Erklärung
14	`app.use(express.json())` – Middleware: liest JSON-Bodies und legt sie in `req.body` ab.
16	`app.use('/api/sessions', sessionsRouter)` – alle Session-Routen hängen unter diesem Präfix.
19-30	Nur im Produktivbetrieb (`SERVE_CLIENT`): `express.static` liefert den gebauten Client aus. Andere `GET`-Pfade wie `/sessions/<id>` bekommen `index.html`, damit ein Reload auch ohne Vite funktioniert (React Router entscheidet dann). In der Entwicklung liefert Vite den Client, dort ist das aus.
33-35	Middleware **nach** allen Routen: Wenn keine Route gepasst hat, kommt die Anfrage hier an → 404 als JSON statt der HTML-Seite von Express.
38-47	**Error-Handler**, erkennbar an den **vier** Parametern `(err, req, res, next)`. `SyntaxError` kommt von `express.json()` bei kaputtem JSON → 400. Alles andere → 500, der Stacktrace wird nur in der Konsole ausgegeben, nie an den Client.

`server/src/config.ts` – `PORT` (Standard 3001, per Umgebungsvariable `PORT` änderbar), `CLEANUP_INTERVAL_MS = 5_000`, `PARTICIPANT_TIMEOUT_MS = 15_000`, dazu `CLIENT_DIST` und `SERVE_CLIENT` für den Produktivbetrieb (nur mit `NODE_ENV=production`). Benannte Konstanten statt «magischer Zahlen». `5_000` ist dasselbe wie `5000`, der Unterstrich dient nur der Lesbarkeit.

`server/src/sendError.ts` – `sendError(res, status, error)` schickt immer `{ error }` mit dem Type `ApiErrorBody`. Ein Ort für die Fehlerform statt zehn.

`server/src/store.ts` – alle Daten und Regeln, **ohne Express**. (Kursthemen: TS, Express)

Zeilen	Erklärung
11-16	`StoredParticipant` – interne Sicht mit der **echten** Stimme und `lastSeen`. Unterscheidet sich bewusst vom öffentlichen `Participant`.
18-24	`Session` mit `createdAt: Date` (intern ein echtes Datum, nach aussen ein ISO-String)
26	`const sessions = new Map<string, Session>()` – die «Datenbank». `Map` statt Objekt: schnelles `get` / `has` / `delete` per Schlüssel, keine Kollision mit geerbten Namen wie `toString`.
36-48	`toSessionState()` – wandelt intern → öffentlich. Zeile 45 ist **die Geheimhaltungsregel**: `vote` nur, wenn aufgedeckt oder wenn es die eigene Person (`viewerId`) ist.
50-56	`listSessions()` – `Array.from(map.values(), mapFn)`
62-79	`createSession()` – 6 Zeichen aus `randomUUID()`. Kurz genug zum Teilen. Weil kurze IDs sich wiederholen können, wird neu gewürfelt, bis die ID frei ist.
105-109	`addParticipant()` – neue Person mit `randomUUID()` und `lastSeen: now`
111-117	`removeParticipant()` – entfernt und prüft danach das Auto-Reveal (die Person war evtl. die letzte ohne Stimme)

Zeilen	Erklärung
120-122	`markAsSeen()` – Heartbeat: `lastSeen = now`
129-138	`removeInactiveParticipants()` – Karteileichen-Strategie, siehe DOKUMENTATION Kapitel 8
144-147	`castVote()` – Stimme setzen, dann Auto-Reveal prüfen
150-157	`revealIfEveryoneVoted()` – `length > 0 &&` verhindert, dass eine leere Session «aufgedeckt» wird (`[].every(...)` ist immer `true`!).
163-168	`resetRound()` – verdecken, alle Stimmen auf `null`
170-177	`changeDeck()` – gleiches Deck → nichts tun, sonst Deck setzen und neue Runde

`server/src/validation.ts` – prüft Request-Bodies. (Kursthema: TS)

Zeilen	Erklärung
13	`ValidationResult<T>` – **Discriminated Union**: entweder `{ ok: true; value: T }` oder `{ ok: false; error: string }`. Nach `if (!result.ok)` weiss TypeScript, dass es `error` gibt, sonst `value`.
15-17	`isObject()` – **Type Guard** (`value is Record<string, unknown>`). Nach dem Aufruf darf man auf Eigenschaften zugreifen. `typeof null === 'object'`, darum zusätzlich `!== null`.
19-22	`isDeckType()` – `Object.hasOwn` statt `in`, weil `'toString' in DECKS` auch `true` wäre (geerbt). Ein Test prüft genau das.
24-36	`validateJoinBody()` – Name muss String sein, wird getrimmt, darf nicht leer und höchstens 30 Zeichen lang sein.
38-50	`validateVoteBody()` – `value` muss vorhanden sein. `null` ist erlaubt (zurückziehen), sonst muss die Karte im **aktiven** Deck sein.
52-57	`validateDeckBody()`

Der Body ist `unknown`, nicht `any`: Mit `unknown` erzwingt TypeScript eine Prüfung vor jedem Zugriff. Mit `any` könnte man ungeprüft `body.name.trim()` aufrufen und der Server würde bei `{ "name": 42 }` abstürzen.

`server/src/routes/sessions.ts` – Router: HTTP ⇄ Store. (Kursthemen: Express, HTTP)

Zeilen	Erklärung
31	`Router()` – ein Mini-App-Objekt, in `app.ts` unter `/api/sessions` eingehängt
34-37	`sendSession()` – baut den `SessionState` für die fragende Person
41-47	`findSessionOr404()` – gibt die Session zurück oder schickt 404. Muster: `if (!session) return`.
49-52	`GET /` – Liste
54-58	`POST /` – neue Session, **201**
60-64	`DELETE /:sessionId` – **204** ohne Body (`res.status(204).end()`)
66-81	`GET /:sessionId` – mit `?participantId=…` zugleich Heartbeat. `req.query.participantId` kann ein String, ein Array oder fehlen, darum `typeof … === 'string'`. Unbekannte ID ist kein Fehler.
83-95	`POST /:sessionId/participants` – validieren (400), hinzufügen, **201** mit `participantId`
97-104	`DELETE …/participants/:participantId` – idempotent, immer 204

Zeilen	Erklärung
106-126	`PUT …/vote` – Reihenfolge der Prüfungen: Session 404 → Teilnehmer 404 → aufgedeckt 409 → Body 400 → speichern → Antwort mit eigener Stimme
128-139	`PUT /:sessionId/deck`
141-155	`POST …/reveal`, `POST …/reset`

`server/src/app.test.ts` – Tests aller Routen mit supertest gegen `app`. Hilfsfunktionen `createSession`, `join`, `getState`, `vote` (Zeilen 15-40) machen die Tests lesbar. `beforeEach(clearSessions)` – jeder Test startet leer. Der Heartbeat-Test (Zeilen 322-354) fälscht mit `vi.useFakeTimers({ toFake: ['Date'] })` nur die Uhr und ruft `removeInactiveParticipants()` direkt auf. (Kursthema: Test)

`server/src/static.test.ts` – testet den Produktivbetrieb: `vi.stubEnv` setzt `NODE_ENV` und `CLIENT_DIST` auf ein kleines Fake-`dist` im Temp-Ordner, `vi.resetModules()` lädt die App neu, weil `config.ts` die Umgebung beim Import liest. (Kursthema: Test, Express)

`server/package.json`: `dev` = `tsx watch src/index.ts` (startet bei jeder Änderung neu), `build` = nur `tsc --noEmit` (Typprüfung, kein JavaScript-Output).

A.4 `client/` – React-App

Einstieg und Konfiguration

Datei	Zweck und wichtige Zeilen	Kursthema
`client/index.html`	Die einzige HTML-Seite (Single Page App). `<div id="root">` (Z. 10) ist der Ort, an dem React rendert. `lang="de"`, Viewport-Meta (Z. 6) für Mobile.	HTML/CSS
`client/vite.config.ts`	Ports aus Umgebungsvariablen `CLIENT_PORT` / `API_PORT` mit Standard 5173/3001 (Z. 7-8), `strictPort: true` (kein stilles Ausweichen), **Proxy** `/api` → Express (Z. 18-20). Testumgebung `jsdom` und Setup-Datei.	Tooling
`client/package.json`	React 19, React Router 7. `build` = `tsc -b && vite build`.	Tooling
`client/tsconfig*.json`	`tsconfig.app.json` für `src/` (DOM-Types, `jsx: react-jsx`), `tsconfig.node.json` für `vite.config.ts`. Überall `strict: true`, `noUnusedLocals`.	TS
`client/public/favicon.svg`	Icon im Browser-Tab	
`client/src/test/setup.ts`	lädt `jest-dom`-Matcher (`toBeInTheDocument`), räumt nach jedem Test auf und leert `localStorage` und `sessionStorage`	Test
`client/src/config.ts`	`POLL_INTERVAL_MS = 1_500` (Tisch), `SESSION_LIST_POLL_INTERVAL_MS = 3_000` (Liste). Der Kommentar erklärt die Abhängigkeit zum 15-s-Timeout des Servers.	HTTP

`client/src/main.tsx` (Kursthemen: React, Context, Routing)

```
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <UserProvider>
      <BrowserRouter>
        <App />
```

- `!` (Non-null Assertion): Wir wissen, dass `#root` in `index.html` existiert.

- Reihenfolge: `StrictMode` aussen (prüft alles), `UserProvider` über dem Router (der Name gilt für alle Seiten), `BrowserRouter` um `App` (die Routen brauchen ihn).
- Z. 6: `import './styles/global.css'` – Vite bindet CSS per Import ein.

`client/src/App.tsx` – die Routen (siehe B.3). `ErrorBoundary` aussen, eine Layout-Route ohne Pfad (Z. 14), darin `index` (= `/`), `sessions/:sessionId` und `*`. (Kursthema: Routing)

API und Hooks

`client/src/api/client.ts` – eine typisierte Funktion pro Route. Komponenten rufen nie selbst `fetch` auf. (Kursthema: HTTP, TS)

Zeilen	Erklärung
23-31	`class ApiError extends Error` mit `status`. `0` = Server gar nicht erreicht. So kann eine Komponente `err instanceof ApiError && err.status === 404` prüfen.
33-37	`RequestOptions` – `method` als Union der vier erlaubten Methoden, optional `body` und `signal`.
39-53	`readErrorMessage()` – liest `{ error }` aus der Antwort. Ist der Body kein JSON (z. B. 502 vom Vite-Proxy, wenn Express nicht läuft), wird eine eigene Meldung erzeugt.
55-79	`send()` – **der wichtigste Teil**. `fetch` wirft nur, wenn **keine** Antwort kam (Netzwerk) → `ApiError(0)`. Eine Antwort mit 404 oder 500 wirft **nicht**, darum Z. 75 `if (!response.ok)`. Abgebrochene Requests (Z. 68-70) werden unverändert weitergeworfen, damit `usePolling` sie erkennt.
82-86	`request<T>()` – **Generic**: Der Aufrufer sagt, welchen Type die JSON-Antwort hat. `as T` ist ein Vertrauensvorschuss an den Server (der Vertrag steht in `shared`).
88-94	`encodeURIComponent` – Sonderzeichen in IDs zerstören die URL nicht
96-148	je eine Funktion pro Route. `getSession` hängt `?participantId=` an (Heartbeat). Funktionen ohne Antwort-Body (`deleteSession`, `leaveSession`) nutzen `send()` statt `request()`, weil 204 keinen JSON-Body hat.

`client/src/hooks/usePolling.ts` – **Custom Hook**, generisch für jede Art von Daten. (Kursthema: Effects)

Zeilen	Erklärung
3-13	`PollingResult<T>` – was der Hook zurückgibt: `data`, `error`, `isLoading`, `refresh`, `setData`
19-22	`fetcher` bekommt ein `AbortSignal`, damit der Hook Requests abbrechen kann
23-27	vier `useState`. `refreshCount` ist nur ein Zähler: Ändert er sich, läuft der Effekt neu.
30	`useRef<AbortController>` – ein Wert, der Renders überlebt, aber **kein** Re-Render auslöst. Ein Ref, weil auch `setData` (ausserhalb des Effekts) den Controller braucht.
32-62	der Effekt: `poll()` sofort, dann `setInterval`. In `poll()` zuerst den alten Request abbrechen (Z. 36), dann neuen starten. Nach dem `await` prüfen, ob inzwischen abgebrochen wurde (Z. 42, 47).
58-61	**Cleanup**: Intervall stoppen, laufenden Request abbrechen
62	Dependencies `[fetcher, intervalMs, refreshCount]` – darum muss `fetcher` stabil sein (`useCallback` in `PokerRoom`, Modul-Funktion in `HomePage`)
64-66	`refresh()` – Zähler erhöhen → Effekt neu → sofort laden
68-75	`setData()` mit `useCallback` – zeigt eine Antwort sofort an und bricht den laufenden Poll ab (sonst flackert der alte Zustand kurz zurück)

`client/src/utils/validateName.ts` – gleiche Regeln wie der Server, gibt eine Fehlermeldung oder `null` zurück. Der Client prüft für schnelles Feedback, der Server zur Sicherheit.

Context

Datei	Erklärung	Kursthema
`context/UserContextValue.ts`	`NAME_STORAGE_KEY = 'scrum-poker:name'` (Z. 7), `UserContextValue` (Z. 9-12), `createContext<UserContextValue | undefined>(undefined)` (Z. 15). `undefined` als Standard = «kein Provider». Eigene Datei, weil React Fast Refresh nur in Dateien funktioniert, die ausschliesslich Komponenten exportieren.	Context
`context/UserContext.tsx`	`UserProvider`: State `name` mit Lazy Init aus `localStorage` (Z. 10), Effekt schreibt jede Änderung zurück (Z. 12-14). Z. 17: In React 19 ist `<UserContext value={…}>` selbst der Provider.	Context, Effects
`context/useUser.ts`	`useContext` + Fehler, wenn kein Provider da ist. Rückgabetyp ohne `undefined` – Komponenten müssen nicht jedes Mal prüfen.	Context

Seiten

`pages/HomePage.tsx` (Kursthemen: State, React)

Zeilen	Erklärung
16-18	Name aus dem Context, Fehler und Gültigkeit **abgeleitet** (kein eigener State)
21	`usePolling(listSessions, SESSION_LIST_POLL_INTERVAL_MS)` (3 s) – `listSessions` ist eine Modul-Funktion, ihre Identität ändert sich nie, darum kein `useCallback` nötig
33-34	`NameForm` kontrolliert (`name` , `onNameChange={setName}`), `CreateSessionButton disabled={!isNameValid}`
41-43	Fehler **zusätzlich** zu alten Daten anzeigen (Liste bleibt sichtbar)
45-56	bedingtes Rendern: Laden, leere Liste, Liste
54	`onDeleted={refresh}` – nach dem Löschen sofort neu laden

`pages/SessionPage.tsx` (Kursthemen: Routing, Effects)

Zeilen	Erklärung
12-16	`participantStorageKey()` – Key in `sessionStorage` , z. B. `scrum-poker:participant:ab12cd` . `sessionStorage` gilt pro Tab: übersteht einen Reload, nicht das Schliessen.
18-30	`resumeOrJoin()` – gibt es eine gespeicherte ID und kennt der Server sie noch (`getSession` mit der ID, zählt zugleich als Heartbeat), geht es mit ihr weiter. Sonst `joinSession()` .
33	`useParams()` liefert `sessionId` aus der URL
38-39	ohne gültigen Namen wird nicht beigetreten
41-44	State: `participantId` , `joinError` , `joinAttempt`
46-94	**Join-Effekt**: beim Mount beitreten oder nach einem Reload fortsetzen, im Cleanup verlassen. `cancelled` -Flag für StrictMode (siehe B.10).
54-55	gespeicherte ID aus `sessionStorage` lesen
67-72	Antwort kam nach dem Cleanup: neu erstellten Teilnehmer wieder verlassen, die gespeicherte ID aber nicht (der zweite StrictMode-Durchlauf macht mit ihr weiter)
73-75	ID merken (Variable `joinedId` , `sessionStorage` , State)
83-93	Cleanup: `cancelled = true` , verlassen (`DELETE`), ID aus `sessionStorage` löschen, State zurücksetzen. Ein Reload baut React nicht ab, darum läuft das Cleanup dort nicht – genau deshalb bleibt man nach einem Reload am Tisch.
94	Dependencies `[sessionId, trimmedName, hasName, joinAttempt]` – ändert sich eines, wird verlassen und neu beigetreten. `joinAttempt` erhöhen = «nochmals versuchen».

Zeilen	Erklärung
96-118	`renderContent()` – je nach Zustand Namensformular, «nicht gefunden», Fehler, Laden oder `PokerRoom`
111-112	`key={participantId}` – neue Person → frische `PokerRoom`-Instanz

`pages/NotFoundPage.tsx` – Text und `<Link to="/">`. Wird für jede unbekannte URL gerendert.

Komponenten

Datei	Zweck	Wichtigste Zeilen	Kursthema
`Layout.tsx`	Header + Platz für die Seite	`<Outlet />` (Z. 12) rendert die aktive Kind-Route	Routing
`ErrorBoundary.tsx`	fängt Render-Fehler	Klassenkomponente, `getDerivedStateFromError` (Z. 17-19). `<Link to="/" reloadDocument>` lädt die ganze App neu und setzt damit auch die Boundary zurück.	React
`ErrorMessage.tsx`	Fehlermeldung	`role="alert"`, optionaler `onRetry`-Button (`onRetry &&`)	React, HTML/CSS
`LoadingIndicator.tsx`	Ladeanzeige	Standardwert `label = 'Wird geladen…'`, `role="status"`	React
`NameForm.tsx`	Namensfeld auf Home	kontrolliertes `<input value onChange>` (Z. 28-29), `event.preventDefault()` beim Absenden (Z. 16), `aria-invalid` / `aria-describedby`	State, HTML/CSS
`CreateSessionButton.tsx`	Session erstellen	`isCreating` / `error` (Z. 13-14), `await createSession()` → `navigate` (Z. 20-22), `try/catch`	State, Routing, HTTP
`SessionList.tsx`	Liste	`[...sessions].sort(...)` – **Kopie** sortieren, `sort()` würde den State verändern (Z. 14). `key={session.id}`	React
`SessionListItem.tsx`	ein Eintrag	`window.confirm` vor dem Löschen, 404 zählt als Erfolg (Z. 37-39), `toLocaleTimeString('de-CH')`, `aria-describedby`	State, HTTP
`SessionNameForm.tsx`	Name bei geteiltem Link	eigener `draft`-State, `validateName(draft)` zeigt dieselbe Meldung wie auf Home (Z. 13, 41-45), erst beim Absenden `setName` (Z. 16-21) → `SessionPage` tritt automatisch bei	State, Context
`SessionNotFound.tsx`	404-Meldung	Link zurück	React
`PokerRoom.tsx`	Herz des Tisches	siehe unten	State, Effects, HTTP
`PokerTable.tsx`	Tischlayout	Personen auf zwei Reihen verteilen (`Math.ceil`, Z. 16), `children` in der Mitte (Z. 32)	React
`ParticipantSeat.tsx`	ein Platz	drei Zustände: leer / verdeckt / aufgedeckt (Z. 22-31). Zwei Kartenseiten übereinander, CSS dreht sie (Flip). Text für Screenreader (Z. 37).	React, HTML/CSS
`RoundResult.tsx`	Ergebnis	`computeResult()` bei jedem Render (Z. 12-13), kein State	State
`RoundControls.tsx`	Aufdecken / Neue Runde	rein über Props gesteuert, Button-Text je nach Ladezustand	React

Datei	Zweck	Wichtigste Zeilen	Kursthema
`DeckSwitcher.tsx`	Deck wählen	`<fieldset>` / `<legend>`, `aria-pressed`, ruft `onChange` nur bei einem anderen Deck auf (Z. 26). `DECK_TYPES` als Liste, weil `Object.keys()` nur `string[]` liefert.	React, TS
`CardDeck.tsx`	Handkarten	`DECKS[deck].cards.map(...)`, `key={value}`, `onClick={() => onCardClick(value)}` (Z. 26)	React
`PlayingCard.tsx`	eine Karte	`<button aria-pressed={selected}>` – echter Button, also tastaturbedienbar	React, HTML/CSS
`ShareLinkButton.tsx`	Link kopieren	`navigator.clipboard.writeText`, Fallback `window.prompt`. Effekt mit Cleanup blendet «Kopiert!» nach 2 s aus (Z. 7-11).	Effects

`components/PokerRoom.tsx` im Detail:

Zeilen	Erklärung
18-22	Props `sessionId`, `participantId`, `onRejoin`
24	`type Action = 'vote' | 'deck' | 'reveal' | 'reset'`
30-33	`fetchSession` mit `useCallback` – stabile Identität, sonst startet `usePolling` bei jedem Render neu
35	Polling = Heartbeat
39-40	`pendingAction`, `actionError`
44-54	`runAction()` – gemeinsames Muster für alle Aktionen: Ladezustand an, Request, `setData` mit der Antwort, Fehler als Meldung, `finally` Ladezustand aus
56-62	404 → `SessionNotFound`; noch keine Daten → Laden oder Fehler
64-76	eigene Person fehlt → «Wieder beitreten»
78-80	abgeleitete Werte: `myVote`, `busy`, `voteCount`
82-86	`handleCardClick` – gleiche Karte nochmals → `null`
92-94	`aria-busy={busy}` am Tisch: Screenreader erfahren, dass eine Aktion läuft
101	«Verlassen» navigiert nur. Das Verlassen selbst macht das Cleanup in `SessionPage`.
110-127	`PokerTable` mit `RoundResult` oder Zähler und `RoundControls` als `children`
129-134	`CardDeck`, gesperrt wenn aufgedeckt oder eine Aktion läuft

CSS

Datei	Wichtig	Kursthema
`styles/global.css`	**Design-Tokens** als CSS-Variablen in `:root` (Farben, Abstände, Radien), `box-sizing: border-box` für alle Elemente, `:focus-visible`, Layout (`.app-main` max. 960 px, zentriert mit `margin: 0 auto`), gemeinsame `.button`-Klassen, `.visually-hidden`	HTML/CSS
`pages/HomePage.css`	**Grid**: eine Spalte, ab 768 px zwei Spalten `minmax(0, 2fr) minmax(0, 3fr)`	HTML/CSS
`components/PokerTable.css`	**Flexbox**: mobil Spalte + Tisch unten (`order: 1`), ab 768 px Reihen mit `flex-wrap`, ovaler Tisch (`border-radius: 999px`), Filz mit `radial-gradient`	HTML/CSS

Datei	Wichtig	Kursthema
`components/ParticipantSeat.css`	Karten-Flip mit `transform: rotateY(180deg)`, `transition`, `backface-visibility`. `prefers-reduced-motion` schaltet die Animation aus.	HTML/CSS
`components/PlayingCard.css`	Hover/gewählt-Zustand, grössere Karten ab 768 px	HTML/CSS
übrige `.css`	je eine Datei pro Komponente, Klassennamen nach BEM (`block__element--modifier`, z. B. `seat__face--front`)	HTML/CSS

Client-Tests

Datei	Technik
`App.test.tsx`	`MemoryRouter initialEntries={[path]}` simuliert eine URL ohne Browser
`pages/HomePage.test.tsx`	`fetch` mit `vi.stubGlobal` ersetzt, `window.confirm` gemockt
`pages/SessionPage.test.tsx`	kleiner Fake-Server `answerLikeTheServer()` (Z. 42-77), zählt alle Requests (`callsTo`). StrictMode-Test rendert mit `<StrictMode>`.
`hooks/usePolling.test.ts`	`renderHook` + `vi.useFakeTimers()` – Zeit vorspulen statt warten
`api/client.test.ts`	prüft URL, Body, Fehlerumwandlung
`context/UserContext.test.tsx`	`localStorage` lesen/schreiben, Fehler ohne Provider
`utils/validateName.test.ts`	Grenzfälle der Namensprüfung

Teil B – Musterantworten auf die typischen Fragen

B.1 «Zeigen Sie Komponente X. Welche Props hat sie, wer rendert sie?»

Vorgehen in der Prüfung: Datei öffnen → `interface …Props` zeigen → mit der Suche (`Ctrl+Shift+F`) nach `<KomponentenName` suchen, um die Stelle zu zeigen, die sie rendert.

Komponente	Props (Type)	Gerendert von (Datei:Zeile)	Was sie macht
`App`	–	`main.tsx:12`	definiert die Routen
`ErrorBoundary`	`children: ReactNode`	`App.tsx:11`	zeigt bei Render-Fehlern eine Meldung
`Layout`	–	`App.tsx:14` (als `element` der Layout-Route)	Header + `<Outlet />`
`HomePage`	–	`App.tsx:15`	Startseite
`SessionPage`	–	`App.tsx:16`	Beitreten/Verlassen, wählt die Ansicht
`NotFoundPage`	–	`App.tsx:17`	unbekannte URL
`NameForm`	`name: string`, `error: string | null`, `onNameChange: (name: string) => void`	`HomePage.tsx:33`	kontrolliertes Namensfeld
`CreateSessionButton`	`disabled: boolean`	`HomePage.tsx:34`	erstellt eine Session und navigiert
`SessionList`	`sessions: SessionSummary[]`, `canJoin: boolean`, `onDeleted: () => void`	`HomePage.tsx:54`	sortiert und rendert die Einträge

Komponente	Props (Type)	Gerendert von (Datei:Zeile)	Was sie macht
`SessionListItem`	`session: SessionSummary`, `canJoin: boolean`, `onDeleted: () => void`	`SessionList.tsx:19`	Beitreten, Löschen
`SessionNameForm`	–	`SessionPage.tsx:98`	Name bei geteiltem Link
`SessionNotFound`	–	`SessionPage.tsx:101`, `PokerRoom.tsx:57`	Session existiert nicht
`PokerRoom`	`sessionId: string`, `participantId: string`, `onRejoin: () => void`	`SessionPage.tsx:111-116`	Polling, Aktionen, verteilt Daten
`DeckSwitcher`	`deck: DeckType`, `disabled: boolean`, `onChange: (deck: DeckType) => void`	`PokerRoom.tsx:96`	Deck wählen
`ShareLinkButton`	–	`PokerRoom.tsx:98`	Link kopieren
`PokerTable`	`participants: Participant[]`, `ownId: string`, `revealed: boolean`, `children: ReactNode`	`PokerRoom.tsx:110`	Plätze + Tischmitte
`ParticipantSeat`	`participant: Participant`, `isOwn: boolean`, `revealed: boolean`	`PokerTable.tsx:20-25`	ein Platz mit Karte
`RoundResult`	`participants: Participant[]`, `deck: DeckType`	`PokerRoom.tsx:112` (als `children` von `PokerTable`)	Durchschnitt, Konsens
`RoundControls`	`revealed`, `canReveal`, `busy`, `revealing`, `resetting: boolean`, `onReveal`, `onReset: () => void`	`PokerRoom.tsx:118-126`	Buttons
`CardDeck`	`deck: DeckType`, `selected: CardValue | null`, `disabled: boolean`, `onCardClick: (value: CardValue) => void`	`PokerRoom.tsx:129-134`	Handkarten
`PlayingCard`	`value: CardValue`, `selected: boolean`, `disabled: boolean`, `onClick: () => void`	`CardDeck.tsx:21-27`	eine Karte
`LoadingIndicator`	`label?: string`	`HomePage.tsx:45`, `SessionPage.tsx:107`, `PokerRoom.tsx:61`	Ladeanzeige
`ErrorMessage`	`message: string`, `onRetry?: () => void`	u. a. `HomePage.tsx:42`, `SessionPage.tsx:104`, `PokerRoom.tsx:107-108`	Fehlermeldung

Beispielantwort für `PlayingCard`: «`PlayingCard` ist eine einzelne Handkarte. Sie hat vier Props: den Kartenwert `value`, ob sie gewählt ist (`selected`), ob sie gesperrt ist (`disabled`) und einen Callback `onClick`. Gerendert wird sie von `CardDeck` in einer `map` über die Karten des aktiven Decks, mit dem Kartenwert als `key`. Sie hat keinen eigenen State: Ob sie gewählt ist, weiss nur `PokerRoom` aus dem Server-Zustand. Sie ist ein echter `<button>` mit `aria-pressed`, damit sie mit Tastatur und Screenreader funktioniert.»

Beispielantwort für `PokerTable` und `children`: «`PokerTable` bekommt neben den Daten auch `children`. Das ist alles, was `PokerRoom` zwischen `<PokerTable>` und `</PokerTable>` schreibt – hier das Ergebnis oder der Zähler und die Buttons. `PokerTable` muss nicht wissen, was in der Mitte

liegt, es legt es einfach in den Filz. Das nennt man Komposition. Ohne `children` müsste `PokerTable` alle Props von `RoundResult` und `RoundControls` durchreichen.»

B.2 «Warum liegt dieser State hier?»

State	Datei:Zeile	Antwort
`name`	`UserContext.tsx:10`	Home und Session-Seite brauchen ihn, sie sind Geschwister unter dem Router. Der gemeinsame Ort ist über dem Router, im Context. Dazu wird er in `localStorage` gespeichert.
`data`, `error`, `isLoading`, `refreshCount`	`usePolling.ts:23-27`	Im Custom Hook, weil Home und Pokertisch dasselbe Polling-Verhalten brauchen. Jeder Aufruf des Hooks hat seinen eigenen State.
`isCreating`, `error`	`CreateSessionButton.tsx:13-14`	Nur dieser Button braucht sie. State so tief wie möglich, so hoch wie nötig.
`isDeleting`, `error`	`SessionListItem.tsx:17-18`	Pro Eintrag eigener State: Löscht man Session A, ist nur A gesperrt. Läge der State in `SessionList`, bräuchte man eine Map von IDs.
`draft`	`SessionNameForm.tsx:11`	Der Name darf erst beim Absenden in den Context, denn jede Änderung des Context-Namens löst in `SessionPage` einen Join aus. Bis dahin ist es ein lokaler Entwurf.
`participantId`, `joinError`, `joinAttempt`	`SessionPage.tsx:41-44`	Sie gehören zum Join-Effekt, der hier läuft. `PokerRoom` bekommt die ID als Prop.
`pendingAction`, `actionError`	`PokerRoom.tsx:39-40`	Alle Aktionen (Karte, Deck, Aufdecken, Reset) teilen sich einen Ladezustand, damit nie zwei gleichzeitig laufen. Darum im gemeinsamen Elternteil aller Buttons (Lifting State Up).
`copied`	`ShareLinkButton.tsx:4`	betrifft nur diesen Button
`hasError`	`ErrorBoundary.tsx:15`	Klassen-State der Boundary

Was bewusst kein State ist: Gültigkeit des Namens, eigene Karte (`myVote`), `busy`, `voteCount`, Ergebnis, sortierte Liste. Alles wird bei jedem Render aus vorhandenem State **abgeleitet**. Faustregel: Was sich aus anderem State berechnen lässt, gehört nicht in einen eigenen State – sonst können zwei Kopien auseinanderlaufen.

B.3 «Welche Routen gibt es, wie kommt die Session-ID in die Komponente?»

«Es gibt drei Routen in `App.tsx`: `/` für die Home-Seite, `/sessions/:sessionId` für den Pokertisch und `*` für alles andere, die Not-Found-Seite. Alle drei hängen unter einer Layout-Route ohne Pfad, damit alle denselben Header haben. `Layout` rendert die aktive Seite an der Stelle von `<Outlet />`.

`:sessionId` ist ein Platzhalter in der URL. In `SessionPage` lese ich ihn mit `useParams()` aus (`SessionPage.tsx:33`). `useParams` gibt `string | undefined` zurück, darum der Standardwert `''`. Danach gebe ich die ID als Prop an `PokerRoom` weiter und brauche sie für alle API-Aufrufe.

Navigiert wird programmatisch mit `useNavigate()`: nach dem Erstellen einer Session (`CreateSessionButton.tsx:22`), beim Beitreten aus der Liste (`SessionListItem.tsx:62`) und beim Verlassen (`PokerRoom.tsx:101`). Für normale Links nehme ich `<Link to="/">`, das wechselt die Seite ohne Neuladen. `BrowserRouter` in `main.tsx` nutzt die History-API, darum funktionieren echte URLs und der Zurück-Button.»

Zusatzfrage «Was passiert bei `/sessions/abc`, wenn es die Session nicht gibt?» → Der Join (`POST …/participants`) antwortet 404, `SessionPage` zeigt `SessionNotFound` (`SessionPage.tsx:100-102`). Die Route selbst passt, denn React Router weiss nichts über Sessions.

B.4 «Verfolgen Sie einen Klick auf eine Karte.»

Kurzfassung zum Auswendiglernen (Details in DOKUMENTATION Kapitel 6):

1. `PlayingCard.tsx:20` – `<button onClick>` → `CardDeck.tsx:26` ruft `onCardClick('5')`.
2. `PokerRoom.tsx:82-86` – `handleCardClick`: gleiche Karte → `null`, sonst `'5'`.
3. `PokerRoom.tsx:44-54` – `runAction`: `setPendingAction('vote')` → alle Buttons gesperrt.
4. `client.ts:128-135` – `castVote()` → `PUT /api/sessions/:id/participants/:pid/vote` mit `{ "value": "5" }`.
5. `vite.config.ts:18-20` – Proxy leitet an Port 3001 weiter.
6. `app.ts:14` – `express.json()` liest den Body.
7. `sessions.ts:106-126` – 404/404/409/400 prüfen, dann `castVote` im Store.
8. `store.ts:144-157` – Stimme speichern, Auto-Reveal prüfen.
9. `sessions.ts:125` – Antwort `SessionState` mit der eigenen Stimme (`store.ts:45`).
10. `PokerRoom.tsx:48` – `setData(antwort)` (`usePolling.ts:68-75` bricht alten Poll ab).
11. Re-Render: `myVote = '5'` → `CardDeck selected` → `aria-pressed="true"`; am eigenen Platz eine verdeckte Karte.
12. Andere Browser sehen die verdeckte Karte beim nächsten Poll (≤ 1.5 s).

B.5 «Welche API-Routen gibt es, welche Methode, warum?»

Tabelle in DOKUMENTATION Kapitel 5 zeigen. Die Kernaussagen:

- **GET** = nur lesen (Liste, Session-Zustand). Sicher und wiederholbar. Kleine Ausnahme: der Heartbeat (`lastSeen`), das ist keine fachliche Änderung.
- **POST** = etwas Neues erstellen, der Server vergibt die ID (Session, Teilnehmer) → 201. Auch für Aktionen ohne eigene Ressource (`reveal`, `reset`).
- **PUT** = einen Wert vollständig setzen (Stimme, Deck). Idempotent: zweimal derselbe Request, gleiches Ergebnis.
- **DELETE** = entfernen → 204. Teilnehmer entfernen ist idempotent.
- Statuscodes: 400 ungültige Eingabe, 404 nicht gefunden, 409 Konflikt mit dem Zustand (Abstimmen nach dem Aufdecken), 500 Serverfehler.

Typische Nachfrage: «Warum ist `vote` kein POST?» → «Weil die Stimme eine Eigenschaft der Person ist, die ich setze. Ich erzeuge keine neue Ressource. Schicke ich zweimal `'5'`, ist das Ergebnis dasselbe – das ist die Definition von idempotent, und dafür ist PUT gedacht.» «Warum `reveal` kein PUT auf `revealed: true`?» → «Ginge auch. Wir haben es als Aktion modelliert, weil «aufdecken» ein Ereignis im Spielablauf ist. Beides ist vertretbar, wichtig ist, dass es konsistent ist.»

B.6 «Was passiert, wenn der Server beim Polling nicht antwortet?»

«Es gibt zwei Fälle. Läuft der Server gar nicht, wirft `fetch` einen Fehler. `send()` in `client.ts:65-73` macht daraus einen `ApiError` mit Status 0 und der Meldung «Server nicht erreichbar.». Über den Vite-Proxy kommt stattdessen eine 502-Antwort ohne JSON, die `readErrorMessage` (`client.ts:48-52`) ebenfalls in «Server nicht erreichbar.» übersetzt.

`usePolling` fängt den Fehler im `catch` (`usePolling.ts:45-48`) und setzt `error`. Wichtig: `data` wird **nicht** gelöscht. Darum bleibt am Tisch der letzte Stand sichtbar, und darüber erscheint «Server

nicht erreichbar. Neuer Versuch läuft …» (`PokerRoom.tsx:107`). Das Intervall läuft weiter. Sobald der Server wieder antwortet, setzt der nächste erfolgreiche Poll `error` zurück auf `null` und die Meldung verschwindet von selbst. Auf der Home-Seite gibt es zusätzlich einen Button «Erneut versuchen», der `refresh()` aufruft.

Antwortet der Server sehr langsam, bricht der nächste Poll den alten Request ab (`usePolling.ts:36`). So stauen sich keine Requests und eine alte Antwort überschreibt nie eine neue.

Bleibt der Server länger als 15 s weg und kommt dann wieder, hat er beim Neustart alle Daten verloren (in-memory) → 404 → «Diese Session existiert nicht (mehr)». War nur das Netz des Clients weg, hat der Server den Teilnehmer nach 15 s entfernt → «Du bist nicht mehr am Tisch» mit «Wieder beitreten».»

B.7 «Warum ist dieser Typ so definiert? Was ginge ohne ihn schief?»

Type	Warum so	Was ohne ihn schiefginge
`DeckType = 'fibonacci' | 'tshirt'`	Union aus Literalen	Mit `string` würde `changeDeck(id, 'tshrit')` kompilieren und erst der Server meldet 400. `DECKS[deck]` könnte `undefined` sein.
`Participant.vote: CardValue | null`	`null` = keine Stimme oder geheim	Mit `vote: CardValue` würde TypeScript nicht erzwingen, den Fall «noch verdeckt» zu behandeln. `RoundResult` würde `null` an `computeResult` geben. Ein optionales `vote?:` wäre unklar: fehlt es oder ist es geheim?
`hasVoted: boolean` getrennt von `vote`	Vor dem Aufdecken muss der Client wissen, **ob** jemand gewählt hat, ohne **was** zu erfahren	Ohne `hasVoted` müsste der Server die Stimme schicken, damit der Client eine verdeckte Karte zeigen kann – dann könnte man sie in den Entwicklertools lesen.
`StoredParticipant` (Server) vs. `Participant` (öffentlich)	intern echte Stimme und `lastSeen`, nach aussen gefiltert	Würde der Server seine interne Struktur direkt senden, gingen geheime Stimmen und Verwaltungsdaten an alle Clients.
`SessionState` / Request-Types in `shared`	ein Vertrag für Client und Server	Benennt jemand im Server `revealed` in `isRevealed` um, würde der Client erst zur Laufzeit `undefined` lesen. So meldet TypeScript den Fehler sofort an beiden Stellen.
`SessionSummary.createdAt: string`	JSON kennt kein `Date`	Mit `Date` würde TypeScript `.getTime()` erlauben, zur Laufzeit ist es aber ein String → Absturz.
`ValidationResult<T>` (Discriminated Union)	`ok: true` mit `value` oder `ok: false` mit `error`	Mit zwei optionalen Feldern könnte man `value` lesen, obwohl die Prüfung fehlschlug.
Body als `unknown` (`validation.ts`)	erzwingt Prüfung vor Zugriff	Mit `any` würde `{ "name": 42 }` beim `.trim()` den Server mit einem 500er abstürzen lassen.
`ApiError` mit `status`	Fehlerart unterscheiden	Ohne Status könnte `SessionPage` 404 («gibt es nicht») nicht von «Server weg» unterscheiden.

Type	Warum so	Was ohne ihn schiefginge
`PollingResult<T>` / `usePolling<T>`	Generic	Ohne Generic wäre `data` `unknown` oder `any` – die Home-Seite wüsste nicht, dass es `SessionSummary[]` ist.
`CreateSessionButtonProps` usw.	Props-Interfaces	Fehlende oder falsch geschriebene Props wären erst im Browser sichtbar.
`type Action` in `PokerRoom`	nur vier erlaubte Werte	`pendingAction === 'revael'` würde kompilieren und nie zutreffen.

B.8 «Welches Problem löst Context? Wann sind Props besser?»

«Context löst **Prop Drilling**: Ein Wert, den viele Komponenten auf verschiedenen Ebenen brauchen, müsste sonst durch jede Zwischenkomponente als Prop durchgereicht werden, auch durch solche, die ihn selbst nicht brauchen. Bei uns ist das der Name: Ihn brauchen `HomePage`, `SessionPage` und `SessionNameForm`. Ohne Context müsste `App` den Namen halten und über die Routen weitergeben.

So ist es gebaut: `createContext` in `UserContextValue.ts`, der Provider in `UserContext.tsx` hält den State und liegt in `main.tsx` über dem Router, und `useUser()` liest ihn mit `useContext`.

Props sind besser, wenn

- der Wert nur ein, zwei Ebenen weit geht – Props zeigen klar, woher Daten kommen;
- der Wert sich oft ändert – jede Änderung eines Context-Werts rendert **alle** Leser neu. Unser Session-Zustand ändert sich alle 1.5 s, darum geht er als Props nach unten;
- eine Komponente wiederverwendbar und einfach testbar sein soll – `PlayingCard` funktioniert ohne Provider.

Context ist gut für wenige, selten wechselnde, «globale» Werte: Name, Sprache, Theme, Login.»

B.9 «Warum braucht es ein Cleanup im `useEffect`?»

«Ein Effekt startet oft etwas, das weiterläuft: ein Intervall, einen Request, einen Timer, eine Mitgliedschaft auf dem Server. Wird die Komponente entfernt oder ändern sich die Dependencies, muss das wieder gestoppt werden. Sonst läuft das Intervall ewig weiter (Memory Leak, unnötige Requests), eine späte Antwort ruft `setState` auf einer Komponente auf, die es nicht mehr gibt, oder es laufen nach einem Wechsel der Dependencies zwei Intervalle gleichzeitig.

Bei uns gibt es drei Cleanups:

1. `usePolling.ts:58-61` – `clearInterval` und laufenden Request abbrechen. Nach dem Verlassen des Tisches wird nicht weiter gepollt, und damit hört auch der Heartbeat auf.
2. `SessionPage.tsx:83-93` – die Session verlassen (`DELETE …/participants/:id`) und die gespeicherte ID aus `sessionStorage` löschen. Der «Verlassen»-Button navigiert nur, dadurch wird die Seite entfernt und das Cleanup läuft. Bei einem Reload läuft es bewusst nicht: Dann bleibt man am Tisch, und die neu geladene Seite macht mit der gespeicherten ID weiter.
3. `ShareLinkButton.tsx:10` – `clearTimeout`, damit «Kopiert!» nicht nach dem Entfernen noch einen State setzt.

Das Cleanup läuft vor dem nächsten Durchlauf des Effekts und beim Unmount.»

B.10 «Was macht StrictMode mit Effects?»

«Im Entwicklungsmodus führt `<StrictMode>` (in `main.tsx:9`) jeden Effekt beim ersten Mount **zweimal** aus: Mount → Cleanup → Mount. Damit findet React Effekte ohne korrektes Cleanup. Im Produktions-Build passiert das nicht.

Bei uns bedeutet das: `SessionPage` startet zwei Joins. Ohne Gegenmassnahme gäbe es zwei Teilnehmer mit demselben Namen. Die Lösung ist das `cancelled`-Flag (`SessionPage.tsx:57-81`):

1. Erster Effekt: `cancelled = false`, `joinSession()` startet.
2. StrictMode ruft das Cleanup auf: `cancelled = true`. `joinedId` ist noch `null` (der Request läuft noch), also wird noch nichts verlassen.
3. Zweiter Effekt: eigenes, neues `cancelled = false`, zweiter `joinSession()`.
4. Die Antwort des **ersten** Joins kommt: `cancelled` ist `true` → dieser Teilnehmer verlässt sofort wieder (Z. 67-72).
5. Die Antwort des zweiten Joins kommt: `cancelled` ist `false` → `setParticipantId`.

Ergebnis: genau ein Teilnehmer, kein Duplikat, keine Leiche. Jeder Effekt-Durchlauf hat sein eigenes `cancelled` (Closure), darum funktioniert das. Ein Test prüft genau diesen Fall (`SessionPage.test.tsx`, «leaves the first participant again in StrictMode»).

Sonderfall Reload: Beide Durchläufe finden dieselbe gespeicherte ID in `sessionStorage` und machen mit ihr weiter. Kommt die Antwort des ersten Durchlaufs nach dessen Cleanup, darf er diese ID **nicht** verlassen, sonst wäre sie für den zweiten Durchlauf weg. Darum `if (id !== savedId)` (Z. 68). Test: «continues as the same participant after a reload (sessionStorage)».

Auch `usePolling` wird zweimal gestartet. Dort bricht das Cleanup den ersten Request ab und stoppt das erste Intervall, es bleibt eines übrig.»

Teil C – Weitere wahrscheinliche Prüfungsfragen

#	Frage	Kurze Antwort
1	Was ist der Unterschied zwischen Props und State?	Props kommen von aussen und sind in der Komponente nur lesbar. State gehört der Komponente, ändert sich mit `setX` und löst ein Re-Render aus.
2	Warum darf man State nicht direkt ändern (`session.revealed = true`)?	React erkennt Änderungen nur über `setState` mit einem neuen Wert. Direkte Mutation löst kein Re-Render aus. Darum `[...sessions].sort()` statt `sessions.sort()`.
3	Wozu dient `key` in Listen?	React erkennt damit, welches Element welches ist, wenn sich die Liste ändert. Wir nehmen stabile IDs (`session.id`, `participant.id`), nie den Index.
4	Was passiert, wenn `fetchSession` ohne `useCallback` definiert wäre?	Jede Render-Phase erzeugt eine neue Funktion → Dependency von `usePolling` ändert sich → Effekt läuft neu → bei jedem Render ein neuer Request und ein neues Intervall.
5	Was ist ein Custom Hook?	Eine Funktion, die mit `use` beginnt und andere Hooks verwendet. `usePolling` und `useUser` kapseln wiederverwendbare Logik.
6	Warum `useRef` für den `AbortController`?	Der Wert muss Renders überleben und von Effekt **und** `setData` erreichbar sein, soll aber kein Re-Render auslösen.
7	Was ist Lazy Initial State?	`useState(() => …)`: Die Funktion läuft nur beim ersten Render. So wird `localStorage` nicht bei jedem Render gelesen.

#	Frage	Kurze Antwort
8	`localStorage` vs. `sessionStorage`?	`localStorage` bleibt nach dem Schliessen des Browsers und gilt für alle Tabs, `sessionStorage` gilt nur für den Tab und übersteht einen Reload. Der Name soll bleiben → `localStorage`. Die `participantId` soll nur einen Reload desselben Tabs überstehen → `sessionStorage`.
9	Warum polling und nicht WebSockets?	Einfacher, passt zu REST, gehört zum Kursstoff. Nachteil: bis 1.5 s Verzögerung und mehr Requests. Für ein Team von 5–10 Personen unproblematisch.
10	Warum 1.5 s Polling-Intervall?	Schnell genug, dass es sich live anfühlt, und weit unter dem 15-s-Timeout (zehn Abfragen Puffer).
11	Was ist CORS und warum braucht ihr es nicht?	Der Browser blockiert Antworten von einem anderen Ursprung (Port zählt mit), wenn der Server es nicht erlaubt. Der Vite-Proxy macht `/api` zum selben Ursprung wie die Seite.
12	Was macht `express.json()`?	Middleware, die JSON-Bodies parst und in `req.body` ablegt. Bei kaputtem JSON wirft sie einen `SyntaxError` → unser Error-Handler → 400.
13	Was ist Middleware?	Eine Funktion `(req, res, next)`, die vor oder nach den Routen läuft. Sie kann die Anfrage verändern, beantworten oder mit `next()` weitergeben.
14	Woran erkennt Express den Error-Handler?	An den vier Parametern `(err, req, res, next)`.
15	Warum ist `app.ts` von `index.ts` getrennt?	Damit Tests die App ohne `listen()` und ohne Timer importieren können.
16	Warum prüft der Server den Namen, obwohl der Client schon prüft?	Der Client ist nicht vertrauenswürdig: Jeder kann mit `curl` Requests schicken. Client-Prüfung = Komfort, Server-Prüfung = Sicherheit.
17	Warum 201 statt 200 beim Erstellen?	201 Created sagt genau, dass eine neue Ressource entstanden ist.
18	Warum 204 beim Löschen?	Erfolg ohne Inhalt. Es gibt nichts zurückzuschicken.
19	Wann kommt 409?	Wenn jemand nach dem Aufdecken noch abstimmen will. Die Anfrage ist gültig, passt aber nicht zum aktuellen Zustand.
20	Wie verhindert ihr, dass man Stimmen in den DevTools sieht?	Der Server sendet vor dem Aufdecken nur `hasVoted`, `vote` ist `null` (`store.ts:45`). Ein Test prüft, dass der Wert nirgends in der Antwort steht.
21	Warum sieht man die eigene Stimme vor dem Aufdecken?	Damit der Client die gewählte Karte aus dem Server-Zustand hervorheben kann, ohne sie lokal zu speichern (Single Source of Truth). Für die eigene Person ist sie kein Geheimnis.
22	Wann wird automatisch aufgedeckt?	Wenn alle (mindestens eine Person) gewählt haben – nach einer Stimme oder wenn die letzte fehlende Person geht.
23	Warum startet ein Deck-Wechsel eine neue Runde?	Alte Karten (z. B. `13`) gibt es im neuen Deck nicht, der Durchschnitt wäre sinnlos.
24	Wie wird der Durchschnitt bei `?` und ☕ berechnet?	Sie zählen nicht zum Durchschnitt, aber zur Anzahl Stimmen. Nur `?` / ☕ → Durchschnitt `–`.
25	Was heisst Konsens bei euch?	Mindestens zwei Stimmen und alle gleich (auch zweimal `?`).
26	Was ist eine reine Funktion, und welche habt ihr?	Gleiche Eingabe → gleiche Ausgabe, keine Nebenwirkungen. `computeResult`, `validateName`, `toSessionState`.

#	Frage	Kurze Antwort
27	Was ist der Unterschied zwischen `unknown` und `any`?	Beide nehmen alles an. Bei `unknown` verlangt TypeScript eine Prüfung vor der Nutzung, `any` schaltet die Prüfung ab. ESLint verbietet `any` bei uns.
28	Was ist ein Type Guard?	Eine Funktion mit Rückgabetyp `value is X`. Nach `if (isObject(body))` weiss TypeScript, dass `body` ein Objekt ist.
29	Was macht `async` / `await`?	`await` wartet auf ein Promise, ohne den Browser zu blockieren. Fehler eines Promise werden mit `try/catch` gefangen.
30	Was bedeutet `void` vor einem Promise?	Wir starten das Promise bewusst, ohne darauf zu warten (z. B. im `onClick`, der nichts zurückgeben soll). Seine Fehler sind innen mit `try/catch` behandelt.
31	Warum wirft `fetch` bei 404 keinen Fehler?	`fetch` wirft nur, wenn keine Antwort kam. Jede HTTP-Antwort ist für `fetch` ein Erfolg. Darum prüfen wir `response.ok` selbst.
32	Wofür `AbortController`?	Um einen laufenden `fetch` abzubrechen: beim Unmount, bei einem neuen Poll oder nach einer Aktion.
33	Was ist der Unterschied zwischen `<Link>` und `<a href>`?	`<Link>` wechselt die Route im Browser ohne Neuladen, State bleibt erhalten. `<a href>` lädt die ganze Seite neu. In der ErrorBoundary wollen wir genau das, darum dort `<Link reloadDocument>`.
34	Was macht `<Outlet />`?	Platzhalter in einer Eltern-Route, an dem die passende Kind-Route gerendert wird.
35	Was ist ein kontrolliertes Formular?	Der Wert des `<input>` kommt aus State (`value`), jede Eingabe geht über `onChange` zurück in den State. React ist die einzige Quelle für den Feldinhalt.
36	Warum `event.preventDefault()` im Formular?	Sonst lädt der Browser beim Absenden (Enter) die Seite neu.
37	Was macht `box-sizing: border-box`?	`width` schliesst Padding und Rahmen ein. Elemente werden nicht unerwartet breiter.
38	Flexbox oder Grid – wann was?	Flexbox für eine Richtung (Reihe von Buttons, Karten), Grid für zweidimensionale Layouts (Spalten der Home-Seite).
39	Was heisst Mobile First?	Grundregeln für kleine Bildschirme schreiben, mit `min-width`-Media-Queries für grössere ergänzen.
40	Wozu `aria-pressed`, `role="alert"`, `role="status"`?	Screenreader erfahren, ob ein Button aktiv ist, und lesen Fehler sofort bzw. Ladehinweise höflich vor.
41	Was passiert bei einem Reload am Tisch?	Beim Neuladen läuft kein React-Cleanup, man bleibt also auf dem Server am Tisch. Die neu geladene Seite findet die `participantId` in `sessionStorage`, prüft per `GET`, ob es sie noch gibt, und macht mit ihr weiter: gleicher Platz, gleiche Stimme, kein Doppel-Eintrag, kein Aufdecken.
42	Was passiert, wenn zwei Personen gleichzeitig das Deck wechseln?	Der Server verarbeitet Requests nacheinander. Der zweite gewinnt, beide sehen beim nächsten Poll denselben Zustand.
43	Was passiert mit den Daten bei einem Server-Neustart?	Alles weg (in-memory `Map`). Bewusst so, es gibt keine Datenbank.
44	Wie testet ihr den Heartbeat ohne 15 s zu warten?	Mit `vi.useFakeTimers({ toFake: ['Date'] })` und `vi.advanceTimersByTime()`, dann `removeInactiveParticipants(Date.now())` direkt aufrufen.
45	Was macht `pnpm --filter`?	Führt einen Befehl nur in einem Paket des Workspaces aus.

#	Frage	Kurze Antwort
46	Warum ein eigenes `shared`-Paket?	Types und `computeResult` nur einmal. Änderungen werden in Client und Server gleichzeitig geprüft.
47	Warum verlasst ihr die Session nicht beim Schliessen des Tabs (`pagehide`)?	`pagehide` feuert auch beim Reload, der Browser unterscheidet die Fälle nicht. Früher war das eingebaut: Ein Reload löschte dann die eigene Stimme, und war man der Letzte ohne Stimme, deckte der Server für alle auf. Ein geschlossener Tab bleibt jetzt bis zu 20 s am Tisch (Heartbeat), aufdecken von Hand geht trotzdem.
48	Was passiert mit zwei Tabs derselben Person?	Jeder Tab hat sein eigenes `sessionStorage`, also eine eigene `participantId`: zwei Einträge am Tisch. Mit `localStorage` wären beide Tabs ein Teilnehmer, und Verlassen in einem Tab würfe auch den anderen raus.
49	Woher weiss die Seite nach einem Reload, ob die gespeicherte ID noch gilt?	Sie fragt den Server (`resumeOrJoin`, `SessionPage.tsx:20-30`). Wurde man inzwischen entfernt (Timeout), fehlt die ID in der Teilnehmerliste und es wird normal neu beigetreten.
50	Was prüfen die E2E-Tests, was die Unit-Tests nicht können?	Das Zusammenspiel im echten Browser mit echtem Server: zwei Personen gleichzeitig, Proxy, Reload, Tab schliessen, Layout ohne horizontales Scrollen, keine Fehler in der Konsole.

Teil D – Demo und Checkliste

D.1 Demo-Drehbuch (max. 3 Minuten, zwei Browserfenster)

Vorbereitung: Fenster **A** (normal, Name «Anna») links, Fenster **B** (privates Fenster, damit `localStorage` getrennt ist) rechts. Beide auf http://localhost:5173. Keine Session offen.

Zeit	Fenster	Aktion	Was ihr sagt
0:00	A	Name «Anna» steht schon da	«Der Name kommt aus dem `localStorage`, gespeichert im UserContext.»
0:10	B	Name leeren → Buttons grau, Meldung. Dann «Ben» eingeben	«Ungültige Namen sperren Erstellen und Beitreten. Der Server prüft zusätzlich.»
0:25	A	«Neue Session erstellen»	«POST `/api/sessions` → 201, dann `useNavigate` auf `/sessions/<id>`. Die ID kommt per `useParams` in die Seite.»
0:40	B	Session erscheint in der Liste (≤ 3 s) → «Beitreten»	«Die Liste pollt alle 3 Sekunden.»
0:55	A	«Link kopieren» zeigen	«Für Teammitglieder, die die Home-Seite nicht nutzen.»
1:05	A	Karte **5** klicken	«PUT `…/vote`. Bei Ben erscheint nur eine verdeckte Karte, der Wert verlässt den Server nicht.»
1:20	B	Karte **8** klicken → automatisch aufgedeckt	«Alle haben gewählt → Auto-Reveal. Durchschnitt 6.5.»
1:35	A	«Neue Runde», beide wählen **3**	«Konsens wird hervorgehoben.»
1:55	B	Deck auf **T-Shirt** wechseln, A wählt S, B wählt L	«Deck-Wechsel startet für alle eine neue Runde. S=2, L=5, Durchschnitt 3.5 → M.»

Zeit	Fenster	Aktion	Was ihr sagt
2:20	B	Tab schliessen	«Ein Reload lässt Ben am Tisch, mit seiner Stimme. Schliesst er den Tab, kommen keine Polls mehr, und der Heartbeat-Timeout entfernt ihn nach höchstens 20 Sekunden.» (Vorher Reload zeigen, dann Tab schliessen.)
2:35	A	DevTools → Network: alle 1.5 s ein `GET …?participantId=`	«Das Polling ist zugleich der Heartbeat.»
2:50	A	«Verlassen» → Home	«Das Cleanup im `useEffect` schickt den DELETE.»

Plan B, falls etwas hängt: Seite neu laden. Wenn der Server abgestürzt ist: Terminal zeigen, `pnpm dev` neu starten, dabei die Fehlermeldung «Server nicht erreichbar.» als Feature erklären.

D.2 Checkliste für den Tag

Am Vortag

- [ ] Repo aktuell: `git pull`, dann `pnpm install`
- [ ] `pnpm test` und `pnpm lint` grün (optional auch `pnpm test:e2e`)
- [ ] Demo einmal komplett mit Stoppuhr durchgespielt
- [ ] Jede Person kann Teil B ohne Unterlagen beantworten
- [ ] Laptop-Akku, Netzteil, ggf. Adapter für den Beamer

30 Minuten vorher

- [ ] Alle anderen Programme schliessen, Benachrichtigungen aus (Fokus-Modus)
- [ ] Terminal 1: `pnpm dev` läuft, `[server]` und `[client]` ohne Fehler
- [ ] Browser A und privater Browser B auf http://localhost:5173, Zoom auf 125 % für den Beamer
- [ ] Alte Test-Sessions löschen (oder Server neu starten)
- [ ] Editor (VS Code) offen mit diesen Dateien in Tabs: `shared/src/index.ts`, `server/src/routes/sessions.ts`, `server/src/store.ts`, `client/src/App.tsx`, `client/src/api/client.ts`, `client/src/hooks/usePolling.ts`, `client/src/context/UserContext.tsx`, `client/src/pages/SessionPage.tsx`, `client/src/components/PokerRoom.tsx`
- [ ] Zeilennummern im Editor eingeschaltet, Schrift gross genug
- [ ] Globale Suche (`Ctrl+Shift+F`) und «Go to Definition» (`F12`) geübt
- [ ] `DOKUMENTATION.pdf` offen als Rückfallebene

Während des Gesprächs

- Erst die Stelle im Code zeigen, dann erklären.
- Ehrlich bleiben: «Das hat die KI so geschrieben, ich verstehe es so: …» ist besser als raten.
- Bei unbekannten Fragen laut denken und im Code nachsehen.

Teil E – Übungsaufgaben zum Selbsttest

Jede Aufgabe: Vermutung aufschreiben → ändern → `pnpm dev` bzw. `pnpm test` → beobachten → mit `git checkout .` zurücksetzen (ohne Git, z. B. aus dem ZIP: vorher eine Kopie des Ordners

anlegen). Die Lösung steht darunter, erst nachher lesen.

1. Polling-Intervall – Setze `POLL_INTERVAL_MS` in `client/src/config.ts` auf `20_000`. Was passiert nach 20 s am Tisch?

> Der Server entfernt dich nach ca. 15–20 s, weil der Heartbeat zu selten kommt. Beim nächsten Poll fehlst du in der Liste → «Du bist nicht mehr am Tisch». Zeigt die Abhängigkeit Polling ↔ Timeout.

2. `useCallback` entfernen – Ersetze in `PokerRoom.tsx:30-33` das `useCallback` durch eine normale Arrow-Function. Beobachte den Network-Tab.

> Bei jedem Render entsteht eine neue Funktion, `usePolling` startet neu. Nach jeder Antwort kommt sofort der nächste Request (Endlosschleife von Requests). ESLint meldet das nicht, weil der Fehler in der Dependency-Logik des eigenen Hooks liegt – darum der Kommentar im Code.

3. Cleanup weglassen – Kommentiere in `usePolling.ts` die Zeilen 59-60 aus. Geh vom Tisch auf die Home-Seite.

> Das Intervall läuft weiter: Im Network-Tab kommen weiter Requests für die Session, und weil sie die `participantId` enthalten, bleibst du als Teilnehmer «lebendig». Test «stops polling … on unmount» schlägt fehl.

4. StrictMode-Schutz entfernen – Lösche in `SessionPage.tsx:67-72` den `if (cancelled)`-Block. Tritt einer Session bei.

> Im Dev-Modus sitzt du zweimal am Tisch. Der StrictMode-Test schlägt fehl.

4b. Reload-Fortsetzung entfernen – Ersetze in `SessionPage.tsx:55` `sessionStorage.getItem(storageKey)` durch `null`. Wähle am Tisch eine Karte und lade die Seite neu (zweites Fenster offen lassen).

> Nach dem Reload tritt eine neue Person «Ben» bei, deine Karte ist weg, und der alte Eintrag bleibt bis zu 20 s als Karteileiche am Tisch. Der Test «continues as the same participant after a reload (sessionStorage)» schlägt fehl.

5. Kopie beim Sortieren – Ändere `SessionList.tsx:14` zu `sessions.sort(...)`.

> Funktioniert scheinbar, verändert aber den State von `usePolling` direkt (Mutation). Erkläre, warum das gefährlich ist (React merkt nichts, andere Leser sehen eine veränderte Liste).

6. Geheimhaltung brechen – Ändere `store.ts:45` zu `vote: participant.vote`.

> Vor dem Aufdecken stehen alle Stimmen in der Antwort (Network-Tab). Die Oberfläche zeigt sie trotzdem verdeckt – zeigt, dass Geheimhaltung auf dem Server passieren muss. Zwei Server-Tests schlagen fehl.

7. Auto-Reveal – Entferne `session.participants.length > 0 &&` in `store.ts:152`. Was passiert in einer leeren Session, wenn die letzte Person geht?

> `[].every(...)` ist `true` → die leere Session ist «aufgedeckt». Die nächste Person sieht einen aufgedeckten Tisch ohne Stimmen.

8. Neue Karte – Füge `'40'` ins Fibonacci-Deck in `shared/src/index.ts` ein.

> Die Karte erscheint sofort im Client **und** der Server akzeptiert sie, weil beide `DECKS` aus `shared` lesen. Durchschnitt funktioniert ohne weitere Änderung.

9. Neuer Deck-Typ – Ergänze `DeckType` um `'powers'` (ohne `DECKS` anzupassen). Starte `pnpm typecheck`.

> TypeScript meldet, dass in `DECKS` der Schlüssel `powers` fehlt (`Record<DeckType, Deck>`). Danach fehlt er noch in `DECK_TYPES` in `DeckSwitcher.tsx` – das prüft TypeScript nicht. Gute Frage: Wie könnte man das absichern?

10. Statuscode – Ändere in `sessions.ts:116` den Code 409 zu 400. Welcher Test schlägt fehl, und warum ist 409 passender?

11. Fehler simulieren – Stoppe während einer Runde nur den Server (im Terminal `Ctrl+C`, dann nur den Client starten mit `pnpm --filter @scrum-poker/client dev`).

> Meldung «Server nicht erreichbar. Neuer Versuch läuft …», der Tisch bleibt sichtbar. Server neu starten → Daten weg → «Diese Session existiert nicht (mehr)». Erkläre warum.

12. Context ohne Provider – Entferne `<UserProvider>` in `main.tsx`.

> Die ErrorBoundary zeigt «Etwas ist schiefgelaufen», in der Konsole steht `useUser() must be used inside <UserProvider>`. Erkläre, warum der Default `undefined` hier hilft.

13. Breakpoint – Ändere in `HomePage.css` `768px` auf `1200px` und verkleinere das Fenster langsam. Wo springt das Layout um?

14. Box-Modell – Entferne `box-sizing: border-box` in `global.css:43`. Wo wird etwas breiter?

> Das Namensfeld und der Button «Neue Session erstellen» haben `width: 100%` plus Padding und Rahmen. Mit `content-box` werden sie breiter als ihre Karte und ragen über den Rand.

15. Neue Route – Füge eine Seite `/hilfe` hinzu (Komponente, Route in `App.tsx`, Link im Header). Erkläre jeden Schritt.

16. Neue API-Route – Baue `GET /api/sessions/:sessionId/result`, die `computeResult()` für eine aufgedeckte Session zurückgibt (sonst 409). Welche Methode, welche Statuscodes, welcher Type in `shared`? Schreibe einen supertest-Test.

17. Test lesen – Erkläre den Test «hides the votes until the cards are revealed» in `server/src/app.test.ts` Zeile für Zeile.

18. Datenfluss rückwärts – Starte bei `RoundResult.tsx` und verfolge, woher `participants` kommt, bis zum `Map`-Eintrag im Server.